Язык Python.
История, Особенности и Возможности

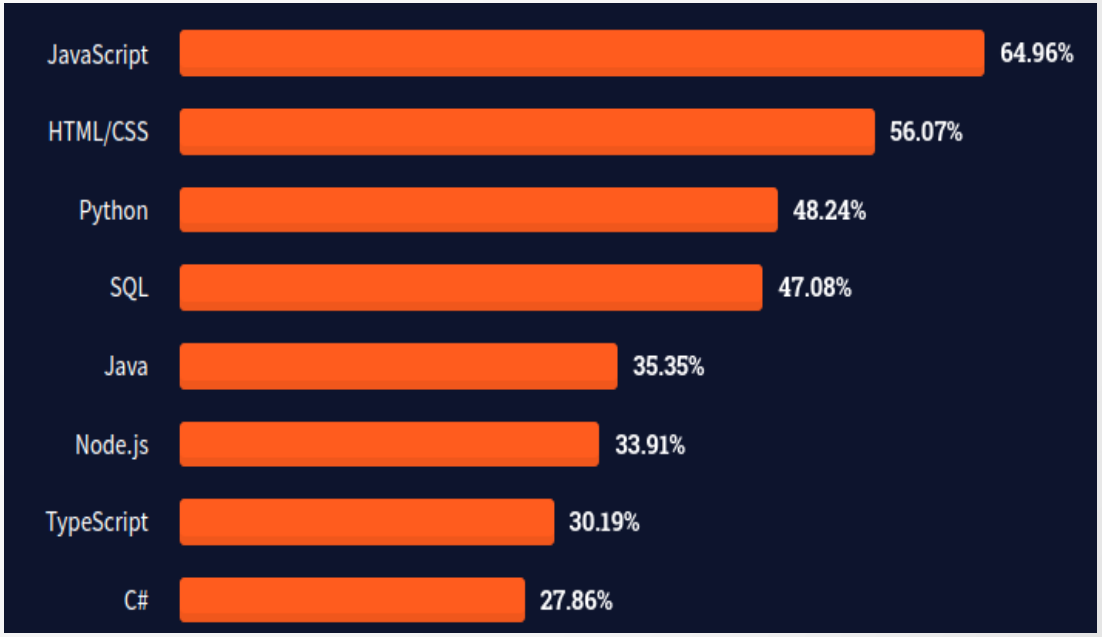
Евгений Борисов

Python. Особенности и возможности

Рейтинг языков программирования TIOBE

Nov 2022	Programming Language
1	Python
2	C
3	Java
4	C++
5	C#

Рейтинг технологий программирования StackOverflow



Python. Особенности и возможности

Проект с историей, существует уже более 30 лет.

Поддержка современных технологий в библиотеках.

Открытое сообщество — доступен всем, над разработкой работают энтузиасты со всего мира.

Универсален — подходит почти для любых решений в области программирования.

Мультиплатформенный — есть реализации почти для всех операционных систем и аппаратных платформ.

Python. Особенности и возможности

Язык программирования общего назначения,

Высокоуровневый,

Императивный, объектно-ориентированный,

Строгая динамическая типизация

Python. Особенности и возможности

Python — высокоуровневый язык программирования.

Высокоуровневый язык программирования — оптимизирован для удобства использования, применяются абстракции — структуры данных, набор вспомогательных функций и т.п.

Низкоуровневый язык — оптимизирован для эффективности выполнения, близок к машинному коду и его конструкциям (Assembler).

Python. Особенности и возможности

Python — императивный язык программирования.

Императивный язык - программа это строго упорядоченный список команд для выполнения.

Декларативный язык - программа это описания результата, который мы хотим получить (SQL)

Python. Особенности и возможности

Python - объектно-ориентированный язык программирования, поддерживает процедурный, структурный и функциональный стиль.

Парадигмы (стили) программирования:

Процедурная — программа строго упорядоченный список команд (Assembler, Shell)

Структурная — программа набор подпрограмм, выполняемый в определённом порядке.

Объектно-ориентированная — программа как набор деталей встроенных друг в друга образующих вместе единый механизм.

Функциональная — программа как суперпозиция математических функций.

Python. Особенности и возможности

Python — строго типизированный язык программирования с возможностью динамической типизации.

Строго типизированный язык - определён ограниченный список типов данных

Динамическая типизация - в процессе выполнения программы переменная может связываться с данными разных типов, объявляем переменную не указываем явно, какой тип данных в ней будет содержаться.

Статическая типизация - тип переменной объявляется явно и в процессе выполнения программы он не меняется.

Python. Особенности и возможности

Существуют реализации Python как интерпретатора, так и компилятора.

Интерпретация — программа оптимизируется и выполняется интерпретатором (специальной виртуальной машиной).

- может выполнятся медленно;
+ независимость от платформы, меньший размер;

Компиляция — программа преобразуется в машинный код (исполняемый файл), который выполняется аппаратной частью непосредственно.

- ограниченная переносимость, ограничения на инструментарий языка;

+ можно добиться оптимального использования вычислительных ресурсов;

IronPython

Python. Особенности и возможности

Python имеет очень много разнообразных библиотек и фреймворков

The Python Package Index (PyPI) is a repository of software
for the Python programming language.

http://pypi.org

Python. Особенности и возможности

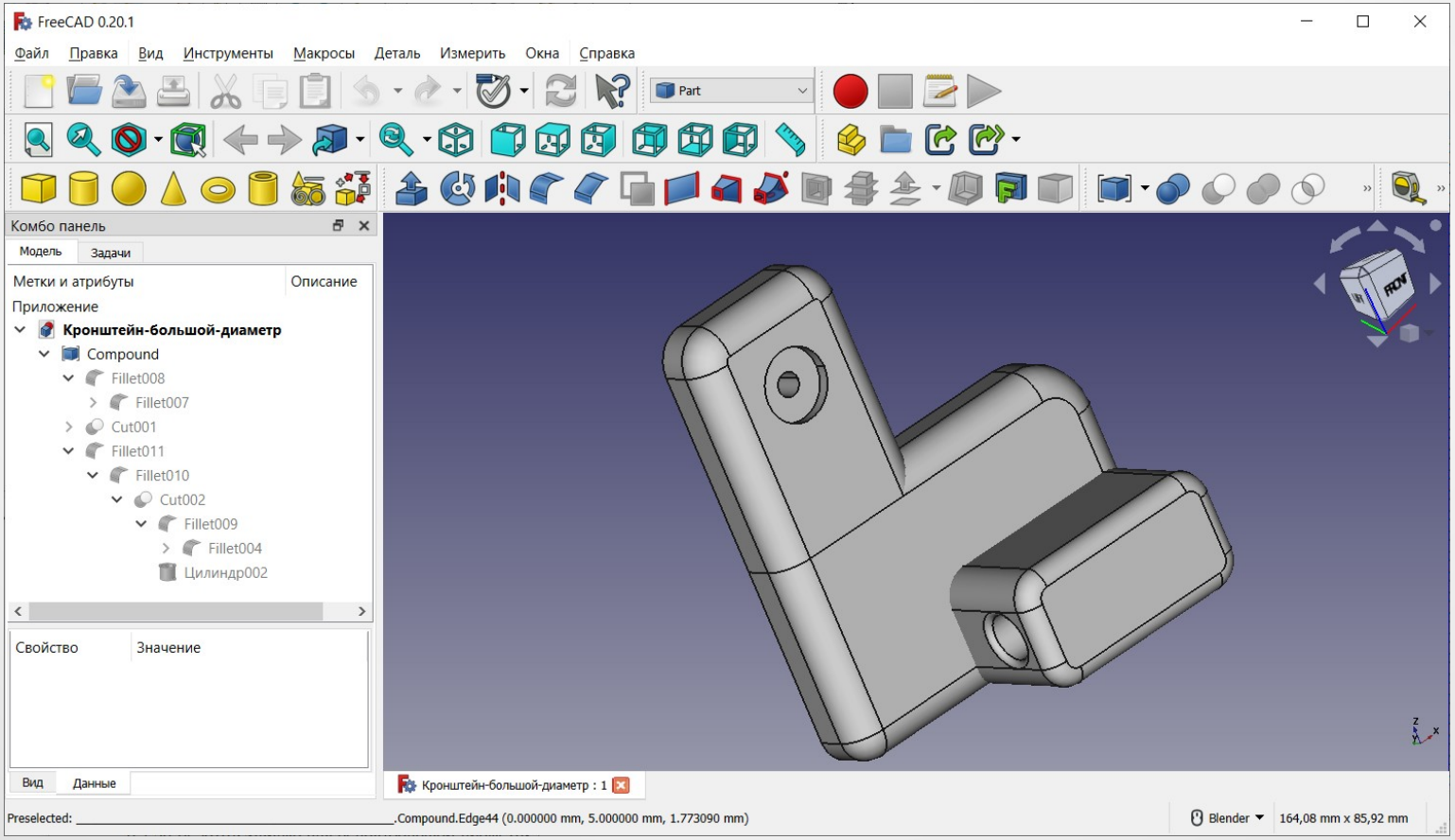
примеры приложений реализованных на Python

World of Tanks - http://tanki.su

Python. Особенности и возможности

примеры приложений реализованных на Python

FreeCad - http://www.freecadweb.org

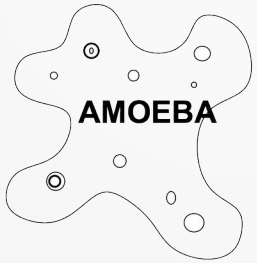


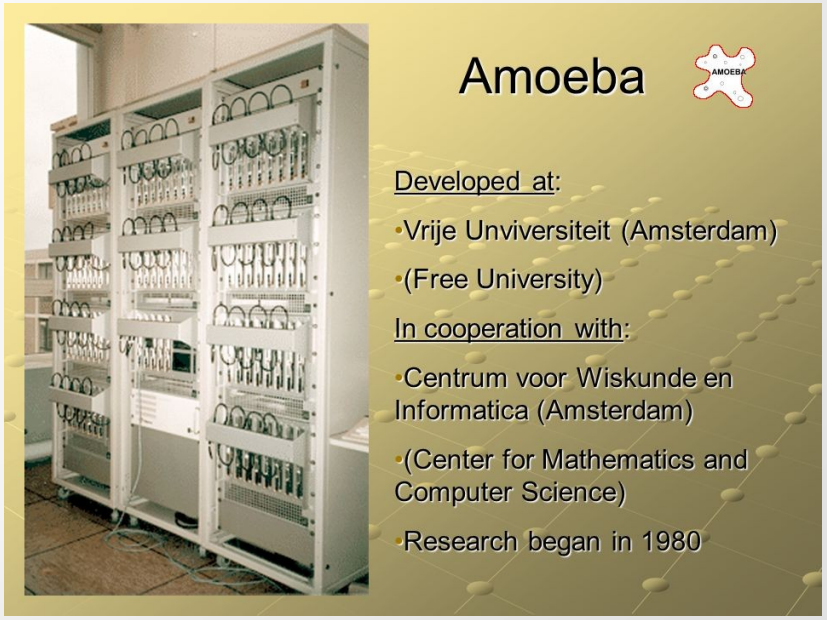
Python. Особенности и возможности

автор первой версии Python
- голландский программист Гвидо ван Россум

центр математики и информатики в Нидерландах,

изначально планировался как язык сценариев для системы Amoeba, начало проекта в декабре 1989 года





Amoeba — открытая микроядерная распределённая операционная система, разработанная группой во главе с Эндрю Таненбаумом в Амстердамском свободном университете. https://www.cs.vu.nl/pub/amoeba/

Python. Особенности и возможности

20 февраля 1991 года через сеть Usenet был опубликован код языка Python .

Так появилась первая версия языка с номером 0.9.0

В языке уже присутствовали:

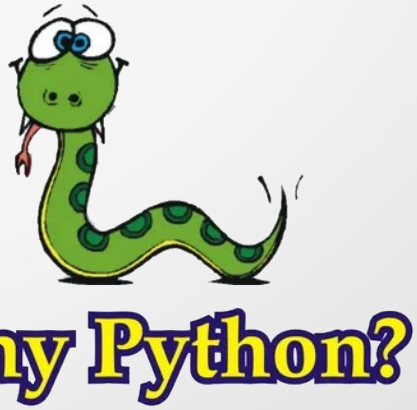
- основные типы данных (list, dict, str),
- поддержка модулей (пакеты подпрограмм),
- классы с наследованием,
- обработка исключений,

В январе 1994 года вышла Python 1.0

Python. Особенности и возможности

Гвидо назвал свой язык в честь комедийного телесериала «Летающий цирк Монти Пайтона»

старый логотип (просуществовал до 2006 года)



Правовые вопросы Python

20 февраля 1991 года был впервые опубликован код языка Python версии 0.9.0.

29 июня 1994 года вышла статья Майкла Маклей из Национального института стандартов и технологий США (NIST) «Если бы Гвидо сбил автобус?»
https://legacy.python.org/search/hypermail/python-1994q2/1040.html
Публикация затронула проблему зависимости Python-сообщества от решений Гвидо

в 1995 году была создана Python Software Foundation — некоммерческая организация, которая должна была отвечать за защиту и развитие языка Python.

Контроль за соблюдением порядка осуществляет «совет руководителей»,
регулярно переизбирается и состоит из пяти человек,

Гвидо ван Россум получил шуточный титул «великодушного пожизненного диктатора»
(BDFL, Benevolent Dictator For Life).

в 2018 году Гвидо отказался от титула BDFL и сделал язык Python полностью независимой технологией.

Python. Особенности и возможности

3 декабря 2008 года выходит Python 3.0, код 3.x и 2.x версии совместим частично

до 2020 2.x и 3.x ветки развивались параллельно

с конца 2020 поддержка ветки 2.x была официально завершена

Python. Особенности и возможности

Python Enhancement Proposals (PEPs)
https://peps.python.org

Предложения по улучшению Python - официальная документация языка.

Позиции из списка PEPs открыто обсуждаются сообществом Python.

PEP 8 Style Guide for Python Code / Руководство по оформлению кода.

PEP 13 Python Language Governance / Список руководителей проекта.

Python. Особенности и возможности

PEP20 - 19 правил по улучшению языка Питон от Тима Петерса,

1. Красивое лучше уродливого.
2. Явное лучше неявного.
3. Простое лучше сложного.
4. Сложное лучше запутанного.
5. Развёрнутое лучше вложенного.
6. Разреженное лучше плотного.
7. Читаемость имеет значение.
8. Особые случаи не настолько особые, чтобы нарушать правила.
9. При этом практичность важнее безупречности.
10. Ошибки не должны замалчиваться.
11. Если не замалчиваются явно.
12. Встретив двусмысленность, отбрось искушение угадать.
13. Должен существовать один - и, желательно, только один – очевидный способ сделать что-то.
14. Хотя этот способ поначалу может быть и не очевиден, если вы не голландец.
15. Сейчас лучше, чем никогда.
16. Хотя никогда часто лучше, чем *прямо* сейчас.
17. Если реализацию сложно объяснить – идея точно плоха.
18. Если реализацию легко объяснить – возможно, идея хороша.
19. Пространства имен – отличная штука! Будем использовать их чаще!

Python: дистрибутивы

CPython

Anaconda (Miniconda)

Python: менеджер пакетов pip

CPython

```
# pip search pep8

# pip install autopep8

# pip list

# pip uninstall autopep8
```

Python: утилиты

показывает места нарушения стиля
pep8 --first main.py

определяет и исправляет нарушения стиля
autopep8 ./ --recursive --in-place -a

форматирует комментарии
docformatter --in-place example.py

универсальная утилита приведения кода к PEP
pyformat

Python: virtual environments

проблема: пакеты определённых версий могут быть несовместимы между собой

решение: виртуальные python-среды

позволяет работать с несколькими версиями python

держать одновременно несколько наборов пакетов разных версий

venv - creation of virtual environments https://docs.python.org/3/library/venv.html

```
# sudo mkdir /opt/venv
# sudo chown -R USER /opt/venv
# cd /opt/venv
# python3 -m venv /opt/venv/jupyter_1
# source /opt/venv/jupyter_1/bin/activate
# pip3 install numpy  ....
```

Python: IDE

IDLE

iPython / Jupyter

PyCharm

Visual Studio Code

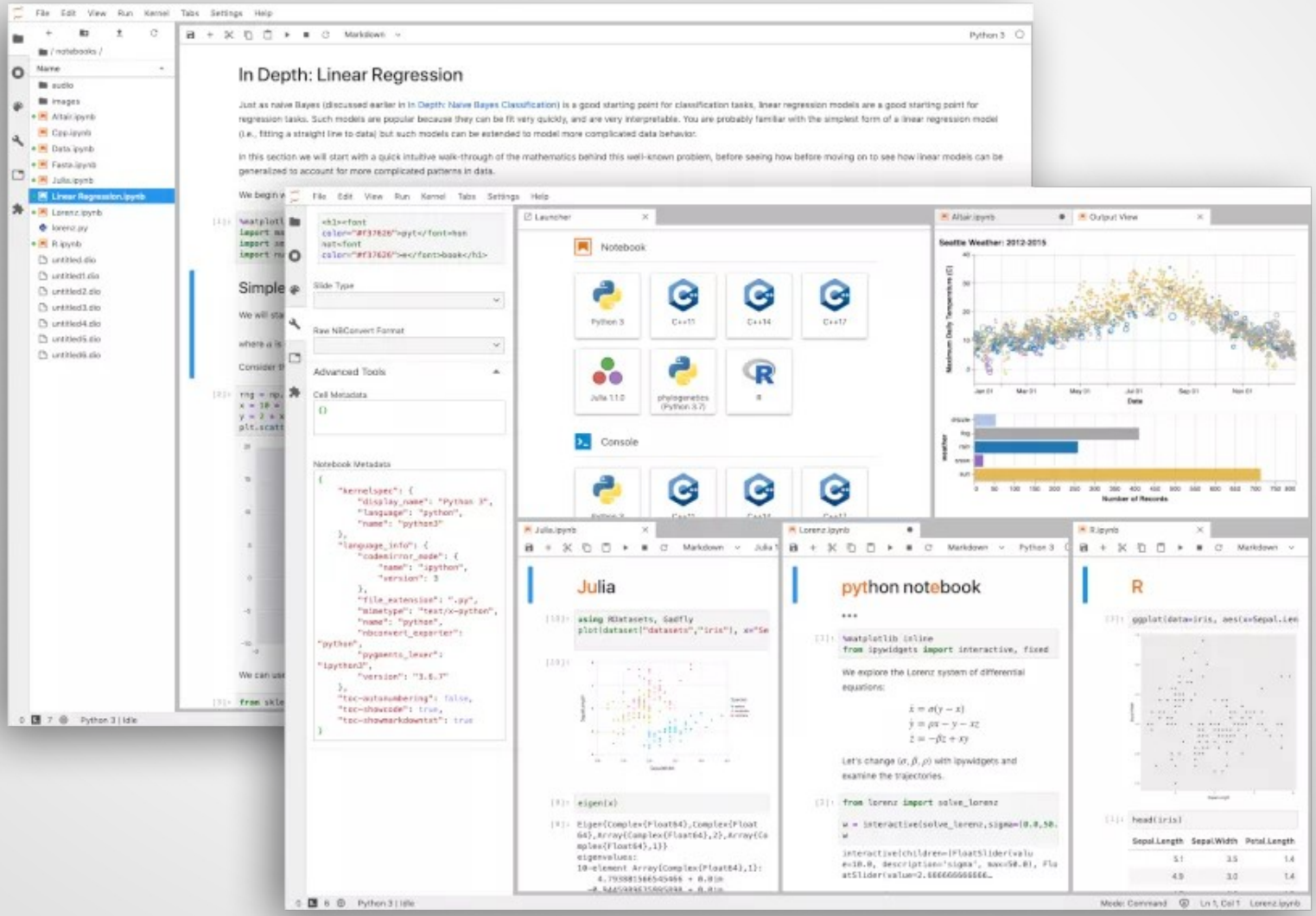
Eclipse + PyDev

Vim

Apache Zeppelin

Python: Jupyter

https://jupyter.org



Python: Google Colab

https://colab.research.google.com/ https://habr.com/ru/post/348058/

Welcome To Colaboratory

File Edit View Insert Runtime Tools Help

Share

Table of contents

Getting started
Data science
Machine learning
More Resources
Machine Learning Examples

Section

+ Code + Text Copy to Drive

Connect Editing

What is Colaboratory?

Colaboratory, or "Colab" for short, allows you to write and execute Python in your browser, with

- Zero configuration required
- Free access to GPUs
- Easy sharing

Whether you're a **student**, a **data scientist** or an **AI researcher**, Colab can make your work easier. Watch Introduction to Colab to learn more, or just get started below!

Getting started

The document you are reading is not a static web page, but an interactive environment called a **Colab notebook** that lets you write and execute code.

For example, here is a **code cell** with a short Python script that computes a value, stores it in a variable, and prints the result:

```
[ ] seconds_in_a_day = 24 * 60 * 60
    seconds_in_a_day
```

```
86400
```

To execute the code in the above cell, select it with a click and then either press the play button to the left of the code, or use the keyboard shortcut "Command/Ctrl+Enter". To edit the code, just click the cell and start editing.

Variables that you define in one cell can later be used in other cells:

```
[ ] seconds_in_a_week = 7 * seconds_in_a_day
    seconds_in_a_week
```

```
604800
```

Colab notebooks allow you to combine **executable code** and **rich text** in a single document, along with **images**, **HTML**, **LaTeX** and more. When you create your own Colab notebooks, they are stored in your Google Drive account. You can easily share your Colab notebooks with co-workers or friends, allowing them to comment on your notebooks or even edit them. To learn more, see Overview of Colab. To create a new Colab notebook you can use the File menu above, or use the following link: create a new Colab notebook.

Python: Google Colab

https://colab.research.google.com/

test1.ipynb

File Edit View Insert Runtime Tools Help All changes saved

+ Code + Text

```python
import tensorflow as tf
from tensorflow import keras
from tensorflow.python.client import device_lib

print('tensorflow:', tf.__version__)
print('keras:', keras.__version__)

if tf.test.is_built_with_cuda():
    print('GPU devices:\n  ',
        [ [x.name, x.physical_device_desc]
          for x in device_lib.list_local_devices()
          if x.device_type == 'GPU' ]
    )
    print('default GPU device:', tf.test.gpu_device_name() )

else:
    print('no GPU device found')
```

```
tensorflow: 1.15.0
keras: 2.2.4-tf
GPU devices:
   [['/device:GPU:0', 'device: 0, name: Tesla T4, pci bus id: 0000:00:04.0, compute capability: 7.5']]
default GPU device: /device:GPU:0
```

Python: Yandex DataSphere

https://cloud.yandex.ru/blog/posts/2020/05/datasphere

File Edit View Run Kernel Snippets Tabs Settings Help

/

Name	Last Modified
catboost_info	seconds ago
train_data.npy	a minute ago
train_labels.npy	a minute ago
yadisk.token	a month ago

test.ml

Code | Migrate instance | Import ipynb | State committed

[1]:
```
from catboost import CatBoostClassifier
import numpy as np
```

[3]:
```
train_data = np.load('train_data.npy')
train_labels = np.load('train_labels.npy')
```

[5]:
```
model = CatBoostClassifier(iterations=1000,
                           task_type="CPU",
                           devices='0:1')
```

[6]:
```
model.fit(train_data, train_labels, verbose=True)
```

```
Learning rate set to 0.003512
0:      learn: 0.6906569        total: 49.8ms   remaining: 49.7s
1:      learn: 0.6881785        total: 50.1ms   remaining: 25s
2:      learn: 0.6871994        total: 50.2ms   remaining: 16.7s
3:      learn: 0.6864504        total: 50.3ms   remaining: 12.5s
4:      learn: 0.6839919        total: 50.4ms   remaining: 10s
5:      learn: 0.6815451        total: 50.5ms   remaining: 8.37s
6:      learn: 0.6791099        total: 50.6ms   remaining: 7.18s
7:      learn: 0.6783759        total: 50.7ms   remaining: 6.29s
8:      learn: 0.6776453        total: 50.8ms   remaining: 5.59s
9:      learn: 0.6757266        total: 50.9ms   remaining: 5.04s
10:     learn: 0.6733133        total: 51ms     remaining: 4.58s
11:     learn: 0.6723679        total: 51.1ms   remaining: 4.21s
12:     learn: 0.6704690        total: 51.2ms   remaining: 3.89s
13:     learn: 0.6685772        total: 51.3ms   remaining: 3.61s
14:     learn: 0.6661971        total: 51.4ms   remaining: 3.38s
15:     learn: 0.6633391        total: 51.5ms   remaining: 3.17s
16:     learn: 0.6609833        total: 51.6ms   remaining: 2.98s
17:     learn: 0.6586417        total: 51.7ms   remaining: 2.82s
18:     learn: 0.6567942        total: 51.8ms   remaining: 2.68s
19:     learn: 0.6549535        total: 51.9ms   remaining: 2.54s
20:     learn: 0.6521563        total: 52ms     remaining: 2.42s
21:     learn: 0.6503330        total: 52.1ms   remaining: 2.32s
22:     learn: 0.6494321        total: 52.2ms   remaining: 2.22s
23:     learn: 0.6471437        total: 52.3ms   remaining: 2.13s
24:     learn: 0.6464625        total: 52.4ms   remaining: 2.04s
25:     learn: 0.6446604        total: 52.5ms   remaining: 1.97s
26:     learn: 0.6428651        total: 52.6ms   remaining: 1.9s
27:     learn: 0.6410765        total: 52.7ms   remaining: 1.83s
28:     learn: 0.6404061        total: 52.8ms   remaining: 1.77s
29:     learn: 0.6381544        total: 52.9ms   remaining: 1.71s
30:     learn: 0.6359178        total: 53ms     remaining: 1.66s
```

0 | 1 | Mode: Edit | Ln 1, Col 1 | c42b3d0c-bbb5-4dd1-9a49-496d67d288eb.ipynb

https://habr.com/ru/article/565086/

Python

Борисов Е.С. Методы машинного обучения. 2026
https://github.com/mechanoid5/ml_lectorium_2026_I

JupyterHub, или как управлять сотнями пользователей Python.
https://habr.com/ru/companies/yandex/articles/353546/

https://www.python.org

https://docs.python.org/3/library/venv.html

https://jupyter.org